How the contract substrate works — economics

How the contract substrate works — economics

> *Explanatory documentation. Canonical economic **policy** lives in* `docs/ECONOMICS-cooperative-commons.md` *(the commons design, its honest wall) and* `DOCTRINE.md` *§ Vine Doctrine / § The Cooperative Commons (the rules and their coexistence). This document explains the mechanics and where each lives in code. Companions:* `docs/HOW-IT-WORKS-architecture.md`*,* `docs/HOW-IT-WORKS-intelligence.md`*.*

The economic layer answers: **why would anyone maintain a route, and what keeps the claims fresh?** Two answers coexist in this substrate, deliberately kept in separate skins (Matt 9:17 — new wine, new bottles):

1. Two economies, one substrate

The Vine Doctrine — canonical staking (slash-to-burn). `DOCTRINE.md` § Vine Doctrine names an existing on-chain reality as the stake substrate: the FDRY token on Solana, the Voltr FDRY-only vault, the stFDRY LP receipt, and the program-enforced 3-day withdrawal cooldown. Depositing FDRY into the vault **is** the act of abiding (John 15:4), made mechanical in `libcontract/src/voltr.zig` (native Solana staking — PDA derivation and deposit-card construction in pure Zig, `libcontract/src/solana_tx.zig` + `libcontract/src/base58.zig`, std-only). Slashed stake *burns to the vault* — NAV-per-share rises for the abiding; nothing is redistributed. Parity invariants in `scripts/build.sh` enforce exactly this and the commons below touches none of them.

The Cooperative Commons (arc-lean) — reward-not-slash. An *alternative* value layer that coexists with (never supersedes) the Vine: usage fees are paid out, in the same asset they arrive in, to the maintainers whose routes were actually **used and still verify**. A stale route is not punished — it simply earns nothing, and the fees its usage generated redistribute to the honest (Luke 10:7 — "the labourer is worthy of his hire"). Policy + Ostrom framing: `docs/ECONOMICS-cooperative-commons.md`.

2. The commons loop, end to end

Every step below is native code; `libcontract/src/epoch.zig` orchestrates one epoch of it, and `libcontract/src/epoch_driver.zig` drives epochs from the ledger.

```
metered call --■ usage row --■ dampened weight --■ epoch settle
                                                          ■
     payout card ■-- USDC split ■-- verified? ■-- re-verify (shape)
```

- **Metering** — `libcontract/src/usage_ledger.zig`: a metered call is one `"usage"` ledger row (a non-terminal event kind, like `evidence`) — the same append-only file as everything else; no side database.
- **Sybil dampening** — `libcontract/src/usage.zig` turns raw usage into the two numbers the reward model needs: the real fee pool (`gross`) and the *dampened weight* a route earns from. Two structural rules remove the wash-trading profit motive: **self-exclusion** (a maintainer paying their own route adds zero weight — but their fee still enters the pool, so a self-dealer donates to the honest) and a **per-payer cap** (weight tracks breadth of *distinct* demand, so faking weight needs many distinct funded wallets paying real fees). The end-to-end test in the source: a wash-trader pays 1000 to itself, earns 0, and the honest maintainer takes the pool.
- **Freshness is checkable** — `libcontract/src/reverify.zig`: the epoch re-runs a route and checks the fresh output still reproduces the committed attestation (`libcontract/src/cas.zig`, content-addressed store). Live data changes every call, so what is hashed is the **structural shape** — a canonical skeleton invariant to values but sensitive to schema. A route that no longer verifies earns nothing this epoch.
- **Reward** — `libcontract/src/reward.zig`, the invariants stated in its header: conservation (`sum(payouts) + treasury == pool` — nothing is burned, contrast slashing); failed routes' fees **redistribute** to the verified; **stake-gated** entry (an unstaked maintainer earns 0 — the Sybil cost that remains, required but never seized). Payouts are in the asset fees arrived in (USDC) — no forced FX risk.
- **Stake is read, not simulated** — `libcontract/src/stake.zig`: the eligibility gate reads the wallet's real bonded stFDRY from chain.

- **Payout is operator-gated** — `libcontract/src/payout.zig` turns epoch results into *broadcastable USDC transfer cards*; a human broadcasts. Money never moves as a side effect of the loop (`libcontract/src/broadcast_card.zig`, `libcontract/src/broadcast.zig`).

3. Paying for the substrate itself — the 402 envelope

Cloud-side declares are metered per call (~$0.001 in USDC). When the caller's wallet is unfunded, the substrate returns a **402 Payment Required** envelope rather than failing silently — and the envelope is itself valid ledger evidence (the doctrine's activation rule counts a 402 as a legitimate outcome of `/contract`). Mechanics in `zig/src/main.zig`: the error classifier maps `payment_required` to the `x402` layer, and the wallet doctor prints both funding rails (Base USDC and Solana USDC — the envelope picks which key signs, ed25519 or secp256k1). Wallet/key custody rules: `DOCTRINE.md` § Wallet & Environment Binding.

4. Value adoption is opt-in property (F1)

Underneath both economies sits the receptor rule (`libcontract/src/capability.zig`, `DOCTRINE.md` § The opt-in receptor): a consumer adopts a producer's value **only through a signed capability grant**. Grants may name non-descendants (cross-lineage reuse survives when explicitly granted); a stranger who merely knows an id cannot adopt. This is what makes "value" an ownable, grantable thing the economics can price at all — and it is enforced by the same signing chain as everything else (`libcontract/src/lineage.zig`).

5. The honest wall — limits, as loud as the design

Stated fully in `docs/ECONOMICS-cooperative-commons.md` § 6; the headlines:

- **Sybil resistance is not formally closed.** Reward-withholding is softer than slashing — a spammer loses nothing they held. Stake-to-enter raises the bar; it does not close the proof. A conscious, non-punitive trade.
- **Usage-gaming is addressed structurally, not formally.** Self-exclusion + per-payer caps remove the *profit motive*; they do not prove gaming impossible.
- **Live vs reference**: the loop above is built and tested end-to-end locally (usage -> weight -> epoch -> reward -> payout cards); on-chain reads (`stake.zig`) are live against Solana; broadcasting remains operator-gated by design, not by gap.

Further reading

- Canonical policy + Ostrom mapping + falsifiable first step: `docs/ECONOMICS-cooperative-commons.md`
- The thesis the whole stack serves: `docs/THESIS-collective-nonparametric-intelligence.md`
- The rules: `DOCTRINE.md` (§ Vine Doctrine, § The Cooperative Commons, § The opt-in receptor, § Wallet & Environment Binding)